stepping stone
IDIBAPS TRAINING PROGRAMME

IDIBAPS RESEARCH COMPUTING

BioData Series

Tools, analysis & perspectives
in biomedical research

2



High performance cluster for beginners: BSC NORD4

Miquel Castany & Leonardo Claudín
Research Computing Platform
IDIBAPS



1. **Introduction to HPC and BSC-Nord4**
   - Overview of cluster's architecture
   - Nord4 specifications and resources
     - Quotas, data cleaning policies and performance impact
   - SSH, login and file structure
   - Data transfer
2. **Node utilization and resource allocation**
   - CPU and memory requests
   - SLURM
   - How to execute interactive jobs
   - Writing and submitting job scripts
3. **Setting up GitLab**
   - Version control
   - Institutional GitLab
     - SOP (Standard Operational Procedure)
     - Singularity images
4. **Good practices in Nord4**
   - Interactive servers
   - Responsible resource allocation
5. **Practical exercise using FastQC**

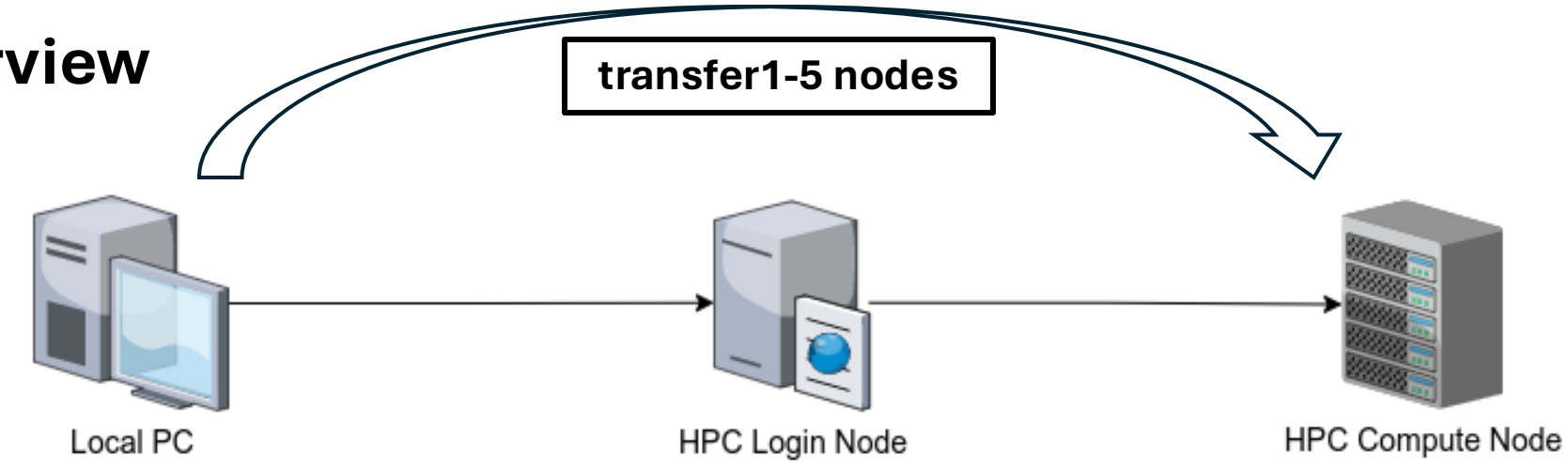


1. **Introduction to HPC and BSC-Nord4**
   - Overview of cluster's architecture
   - Nord4 specifications and resources
     - Quotas, data cleaning policies and performance impact
   - SSH, login and file structure
   - Data transfer
2. **Node utilization and resource allocation**
   - CPU and memory requests
   - SLURM
   - How to execute interactive jobs
   - Writing and submitting job scripts
3. **Setting up GitLab**
   - Version control
   - Institutional GitLab
     - SOP (Standard Operational Procedure)
     - Singularity images
4. **Good practices in Nord4**
   - Interactive servers
   - Responsible resource allocation
5. **Practical exercise using FastQC**

1. Introduction to HPC and BSC-NORD4

Overview of the cluster's architecture and specifications

HPC overview



BSC-NORD4

- n4login1
- n4login2
- n4login3

48 cpus/node

240 nodes **lowmem**:

- **98GB**/node ≈ 2GB/ cpu

120 nodes **highmem**:

- **384GB**/node ≈ 8GB/cpu

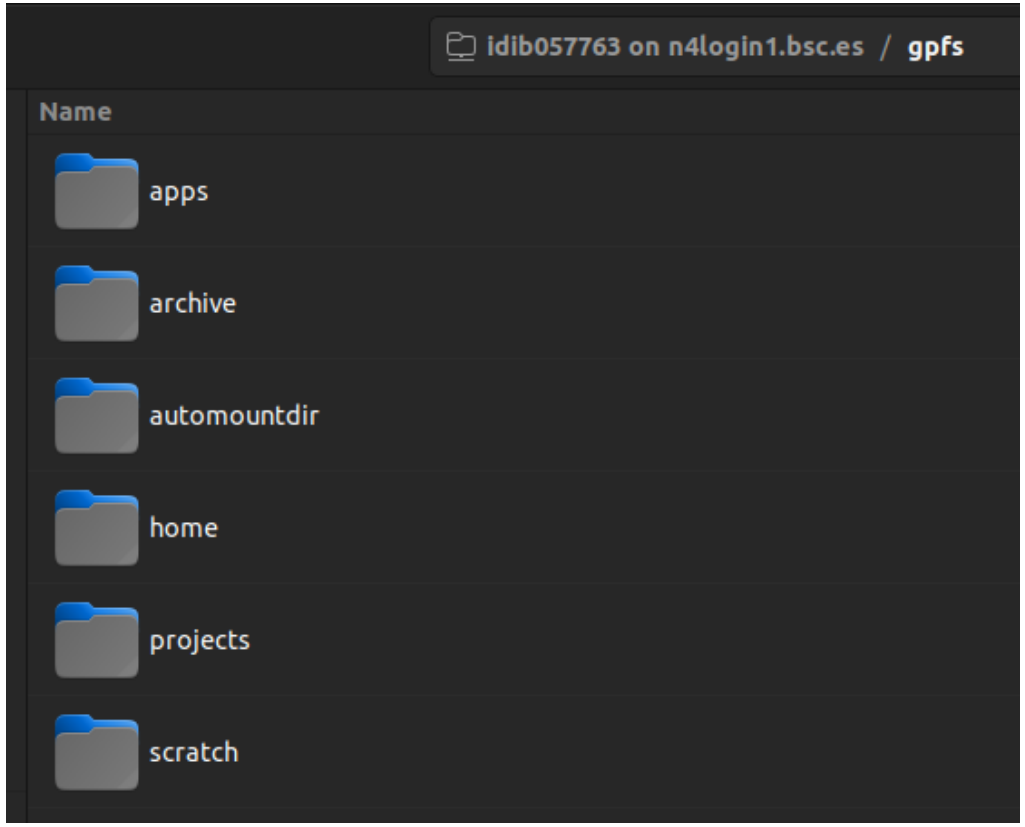
+ 5 transfer nodes for data transfer.

Login nodes are intended to establish connection not to compute!

GPFS* filesystem

*General Parallel File System

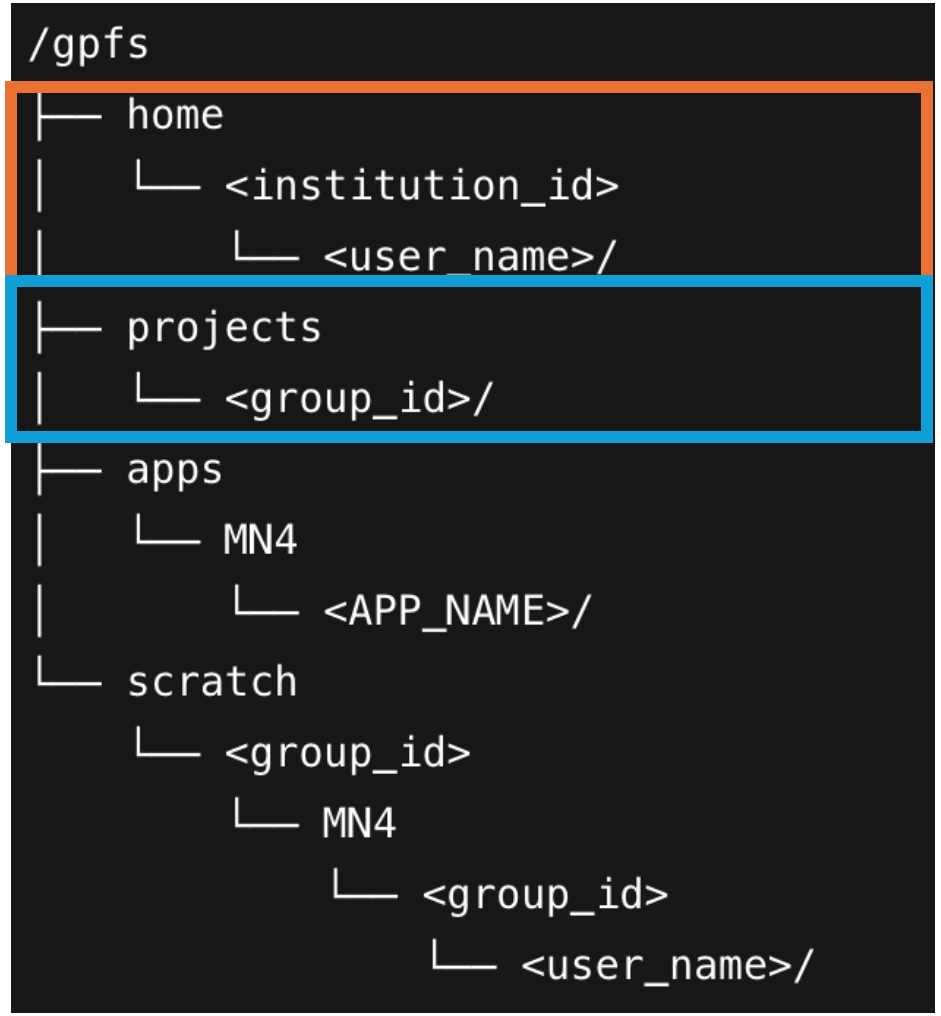
The filesystem root of Nord4 is mounted on /gpfs/




```
/gpfs
├── home
│   └── <institution_id>
│       └── <user_name>/
├── projects
│   └── <group_id>/
├── apps
│   └── MN4
│       └── <APP_NAME>/
└── scratch
    └── <group_id>
        └── MN4
            └── <group_id>
                └── <user_name>/
```

1. Introduction to HPC and BSC-NORD4

Nord4 architecture and specifications



/gpfs/home/<institution_id>/<user_name>/

- **Default directory** when logging to cluster with ssh.
- It is **Private** to each user.
- **10 GB.**
- Here you can store:
  - **Personal data**
  - **Documents**
  - **Scripts**

- A **backup** is performed **daily**.

/gpfs/projects/<UNIXGROUP>/

- **Shared** directory only by group members. It should contain:
  - Data.
  - Execution folders.

- It is not intended for long-term storage.
- **/gpfs/projects** is backed up periodically.
- The quota of these job queues can be consulted running **bsc_quota** in the terminal.
- Beware of permisions while working with shared files/folders.

1. Introduction to HPC and BSC-NORD4

In Unix-like systems (e.g., Linux), **chmod** changes the **permissions (access modes)** of files or directories.

Permissions apply to three classes of users:

- **u** = user (owner)
- **g** = group
- **o** = others

Two main notations for defining permissions:

- **Symbolic**, e.g. chmod u=rwx,g=rx,o=r file
- **Octal**, e.g. chmod 754 file
- (7 = 4+2+1 → rwx; 5 = 4+0+1 → r-x; 4 = 4+0+0 → r--)

- **Common options:**
- -c, --changes → report only when a change is made
- -f, --silent, --quiet → suppress most error messages
- -v, --verbose → output a diagnostic for each file processed
- -R, --recursive → change files and directories recursively

- Examples
  - (symbolic) chmod u=rwx,g=rx,o=r myfile
  - (symbolic) chmod g=u –R path/to/folder
  - (octal) chmod 754 myfile
  - chmod 644 file.htm → owner can read/write; group & others read only
  - chmod -R 755 myfiles → recursive; user rwx, group & others r-x
  - chmod u=rw example.jpg → owner read/write; group & others unchanged

1. Introduction to HPC and BSC-NORD4

Nord4 architecture and specifications

```
/gpfs
├── home
│   └── <institution_id>
│       └── <user_name>/
├── projects
│   └── <group_id>/
├── apps
│   └── MN4
│       └── <APP_NAME>/
└── scratch
    └── <group_id>
        └── MN4
            └── <group_id>
                └── <user_name>/
```

/gpfs/apps/MN4/<APP_NAME>/

- Software installed in Nord4.
- All software should be loaded before running any process.
  - module load <APP_NAME/VERSION>
  - Example:

    ```
    module load gcc pcre2 R/4.3.0
    ```

- Users do not have permission to modify any content.

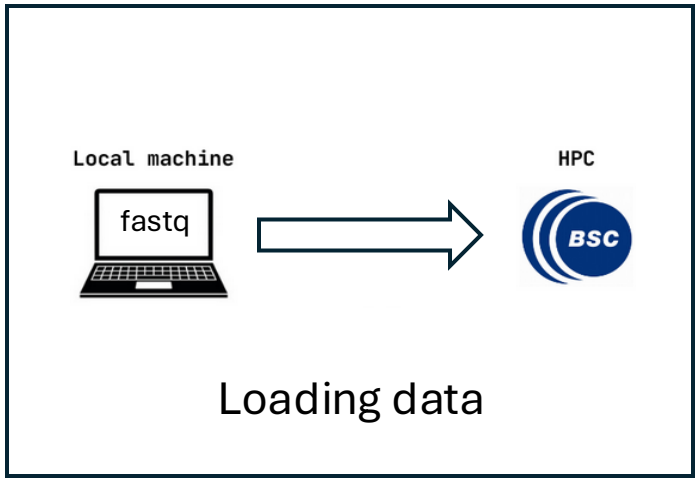
/gpfs/scratch/<UNIXGROUP>/MN4/<UNIXGROUP>/<user_name>

- **Temporary storage** for:
  - **/tmp** files.
  - **/work** folders.
  - Intermediate files generated by job execution (that are not results).

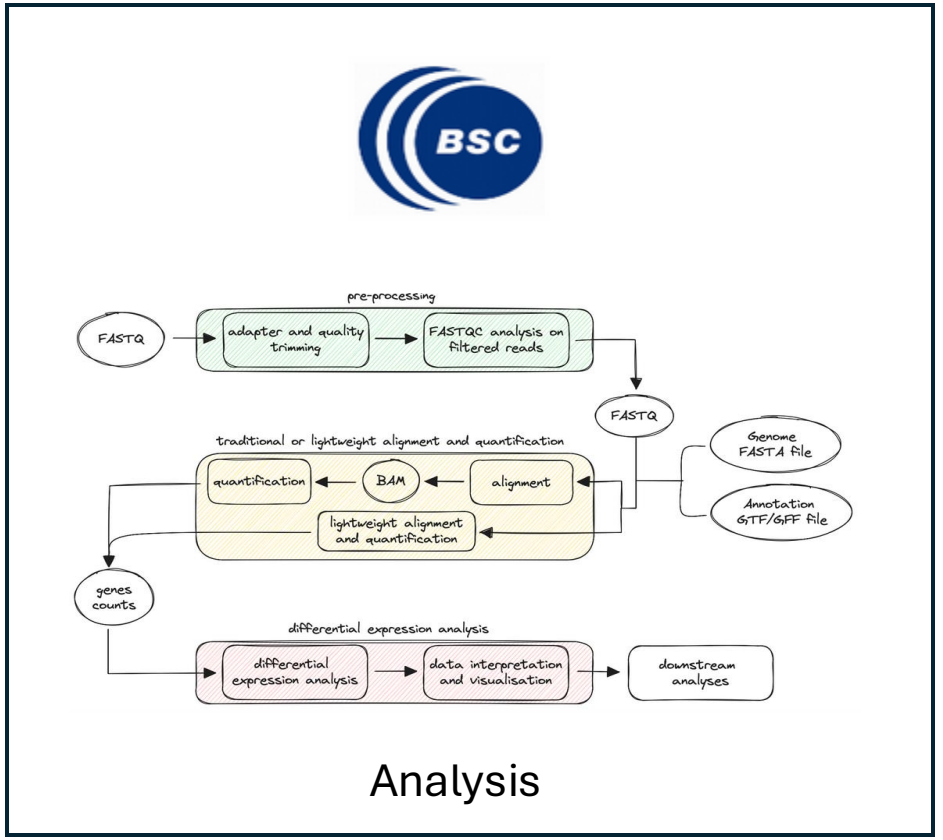
- It is **not backed up**.
- Users are responsible for freeing up space.

Data maintenance 1

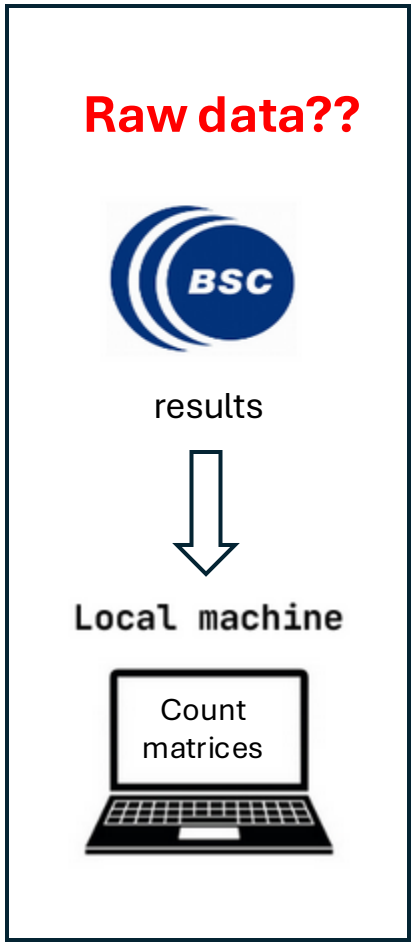
Step 1



Step 2



Step 3



Data maintenance 2

`bsc_quota`

Filesystem	Type	Usage	Quota	Limit	In doubt	Grace	Files	In doubt
gpfs_home	USR	1.45 GB	80.00 GB	84.00 GB	0.00 KB	None	11185	0
gpfs_projects	GRP	9.88 TB	64.60 TB	67.83 TB	5.56 GB	None	164427	289
gpfs_scratch	GRP	0.00 KB	2.00 TB	2.10 TB	0.00 KB	None	13	0

/gpfs/**home** = 10 GB

/gpfs/**projects** = execution directories

/gpfs/**scratch** = temporary files

HUB tecnològic

IDIBAPS

- Compute nodes
- GPU's
- Storage for each group
- More...

To be operating in **≈ 1 year** (being pessimistic)

How to access Nord4

From the terminal:

```
ssh <user_name>@<login_node>.bsc.es
```

Example:

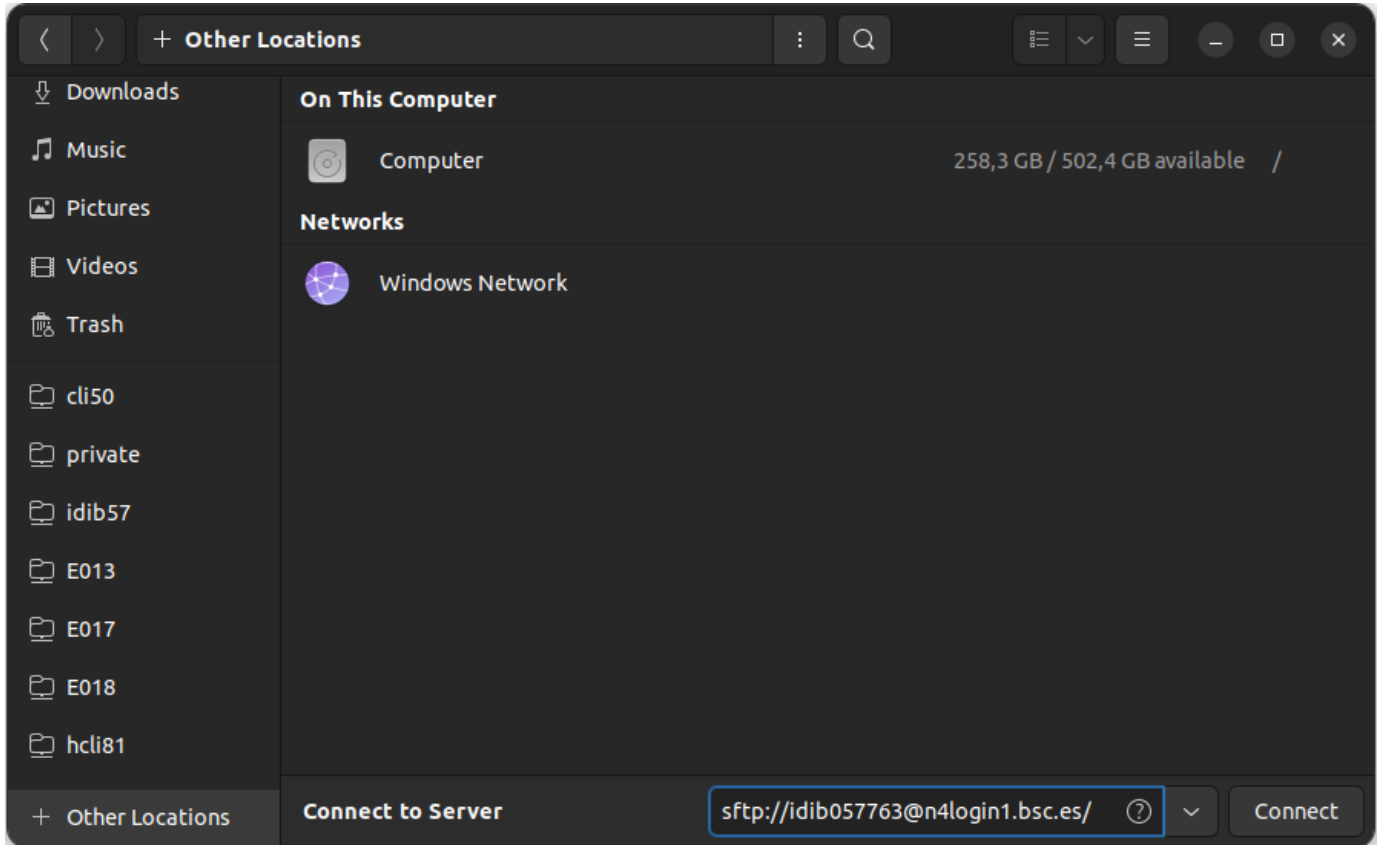
```
ssh idib057763@n4login1.bsc.es
```



From nautilus (Ubuntu's file explorer):

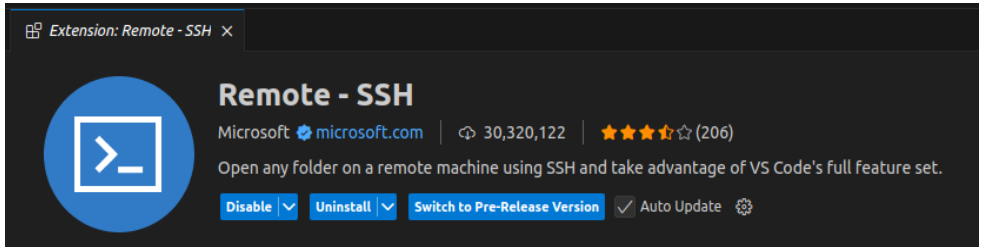
`sftp://<user_name>@<login_node>.bsc.es`

Example:
`sftp://idib057763@n4login1.bsc.es/`

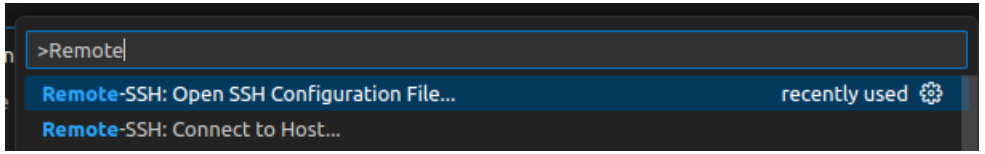


From Visual Studio Code

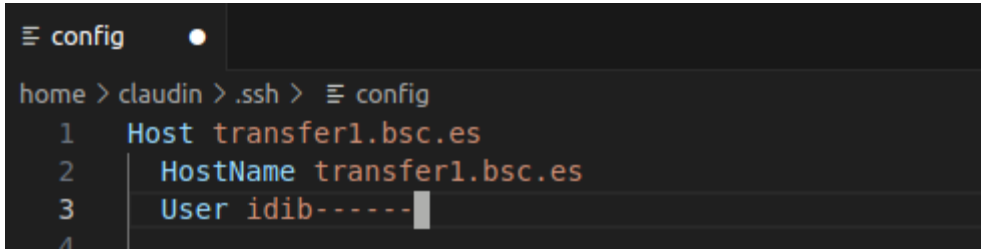
(1) Install Remote – SSH extension



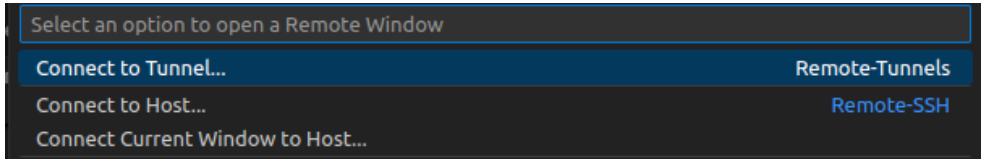
(2) Open SSH Configuration file



(3) Configure node details and user credentials



(5) Start remote connection to defined host



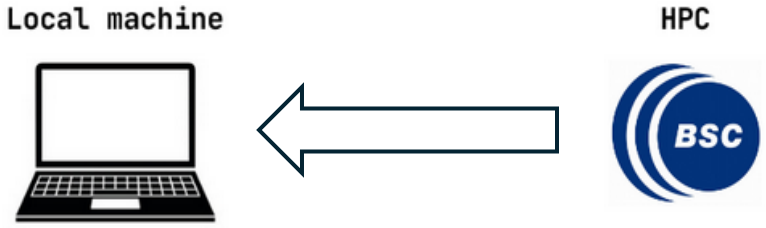
How to transfer data

Use **transfer nodes** to transfer data to/from the HPC.



Execute in local:

```
rsync -av --progress --partial --append local/path/files
<user_name>@transfer1.bsc.es:/path/to/remote/directory
```



Execute in local:

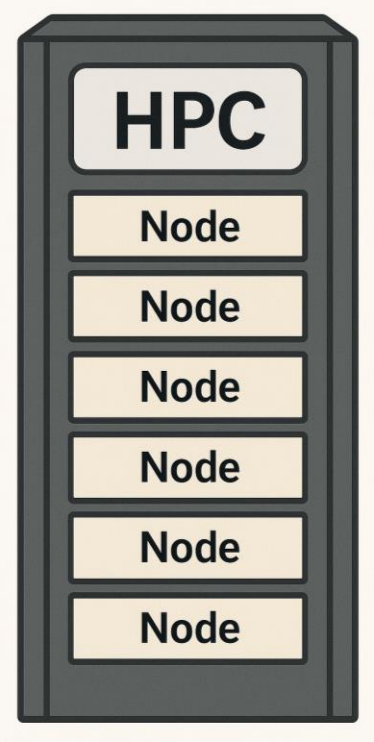
```
rsync -av --progress --partial --append
<user_name>@transfer1.bsc.es:/path/to/remote/files
/your/local/destination/
```



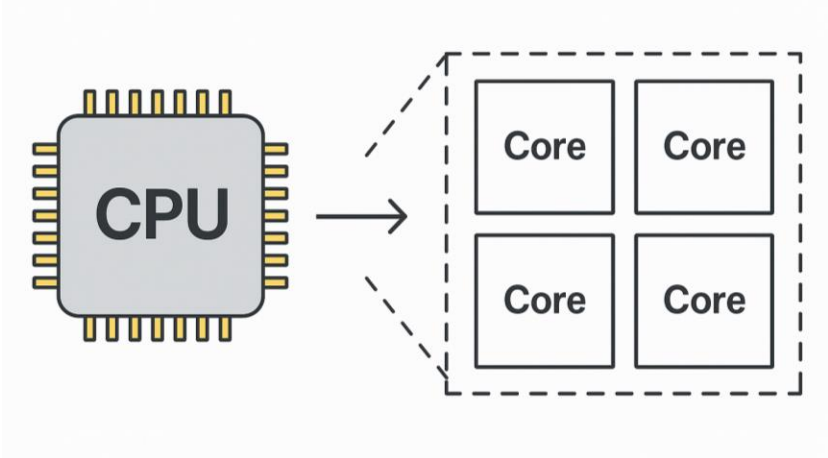
1. **Introduction to HPC and BSC-Nord4**
   - Overview of cluster's architecture
   - Nord4 specifications and resources
     - Quotas, data cleaning policies and performance impact
   - SSH, login and file structure
   - Data transfer
2. **Node utilization and resource allocation**
   - CPU and memory requests
   - SLURM
   - How to execute interactive jobs
   - Writing and submitting job scripts
3. **Setting up GitLab**
   - Version control
   - Institutional GitLab
     - SOP (Standard Operational Procedure)
     - Singularity images
4. **Good practices in Nord4**
   - Interactive servers
   - Responsible resource allocation
5. **Practical exercise using FastQC**

CPU and memory requests

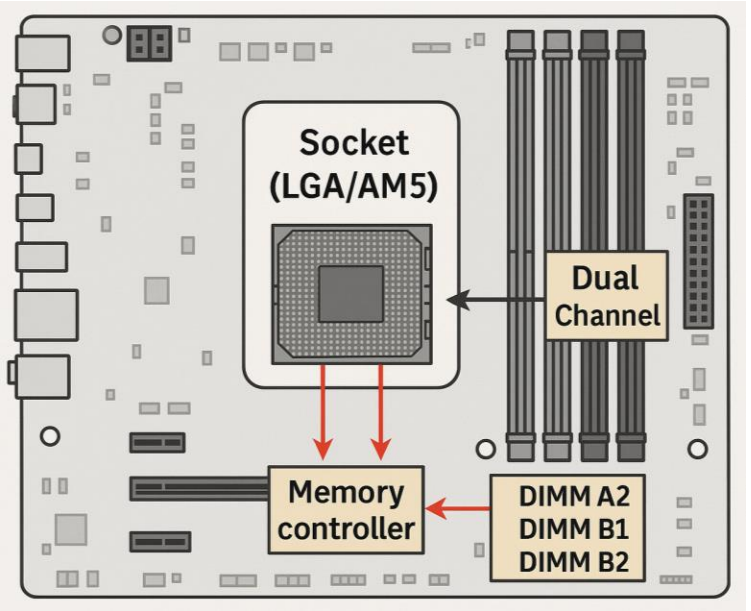
Understanding nodes, cpus, cores...



Housing 360 nodes



Cores can be allocated using:
--ntasks=<number>
--cpus-per-task=<number>
Where:
The total number of cores assigned to the job is:
total_cores = ntasks × cpus_per_task = RAM



We have standard node (cores requested comes with its 2 GB RAM) or high mem nodes (8 GB RAM) selected by
--constraint=highmem

SLURM batch processing

SLURM: Simple Linux Utility for Resource Management

SLURM is a job scheduler:

1. Allocates access to computational resources.
2. Provides a framework for starting, executing and monitoring work.
3. Manages queues of pending jobs.

How to use SLURM	
Interactively	Scripts
`salloc`	#SBATCH directives

Interactive jobs

Interactive jobs = executed in the terminal
Allocating resources through `salloc` command.



We want to run jobs on compute nodes, not in login nodes.

Basic syntax:

```
salloc –n<number_of_nodes> –c<number_of_cores> -A <UNIXGROUP> -q <queue>
```

Example:

`salloc –n1 –c4 -A idib57 -q debug` # interactive job in 1 single node, 4 cpus and running in debugging queue (user belong to UNIXGROUP idib57).

Understanding queues

`bsc_queues`

```
>>>>>>>>>>>>>>>>>>>> ACCOUNT: idib57 <<<<<<<<<<<<<<<<<<<<
    QUEUE NAME         MAX TIME    MAX PROC   DESCRIPTION
         debug         02:00:00         768   Debug jobs with limitations in number of jobs running, time and number of nodes used
   interactive         02:00:00           4   Interactive jobs
           cli (D)  3-00:00:00        1200   -
```

3 queues:

- On partition main:
  - cli: 25 nodes/job, 1500 jobs in queue, 3 days
  - debug: 16 nodes/job, 366 jobs in queue, 1 job running (only 2h!!!)
- On partition interactive:
  - interactive: 4 cores, 366 jobs in queue, 1 job running (only 2h!!!)

Interactive jobs

Inside a `salloc` allocation we can:

- Load modules
- Run code
- Run scripts
- Open R terminal
- Open Python terminal
- …

To check how our job is doing we can use `squeue` command.
Output example:

```
   JOBID PARTITION     NAME     USER ST       TIME  NODES NODELIST(REASON)
32597260 interacti interact idib0577  R       7:44      1 login4
32597257      main interact idib0577 PD       0:00      1 (QOSMaxJobsPerUserLimit)
32597261      main interact idib0577  R       0:06      1 s01r2b45
```

To cancel a job, we have 2 options:

- `scancel <JOBID>` # Interrupts a specific job.
- `scancel –u <user_name>` # Interrupts all jobs (runing or not) from a user.

Slurm parameters

Slurm directives allow users to request and coordinate HPC resources. Slurm acts both as **manager** of HPC resources and **executer** of our jobs on the system.

```
#!/bin/bash
#SBATCH --job-name=<jobname>
#SBATCH --output=<out_file.out>
#SBATCH --error=<error_file.err>
#SBATCH --account=<UNIXGROUP>
#SBATCH --nodes=<number_o_fnodes>
#SBATCH --cpus-per-task=<number_of_cpus>
#SBATCH --time=<d-hh:mm:ss>
#SBATCH --qos=<queue>
```

Script extension:
.sh OR *.sbatch* file types

Job submission:
sbatch OR *srun* commands

Job variables and format specifiers

Format specifier	Variable ($)	Description
%j	*SLURM_JOB_ID*	Job ID (allocation)
%A	*SLURM_ARRAY_JOB_ID*	Job array's master (allocation) job ID
%a	*SLURM_ARRAY_TASK_ID*	Job array index (task ID within the array)
%u	*SLURM_JOB_USER*	Username running the job

These variables can be used in sbatch scripts.
They are very usefull for generating stderr and stdout logs and debugging.

Job states

Job state code (ST)	Description
Pending (PD)	Waiting for resource allocation
Running (R)	Active and running
Completing (CG)	Finishing, but some process are still active. This code appears when cancelling.
Completed (CD)	Terminated with exit_code=0
Failed (F)	Terminated with exit_code != 0

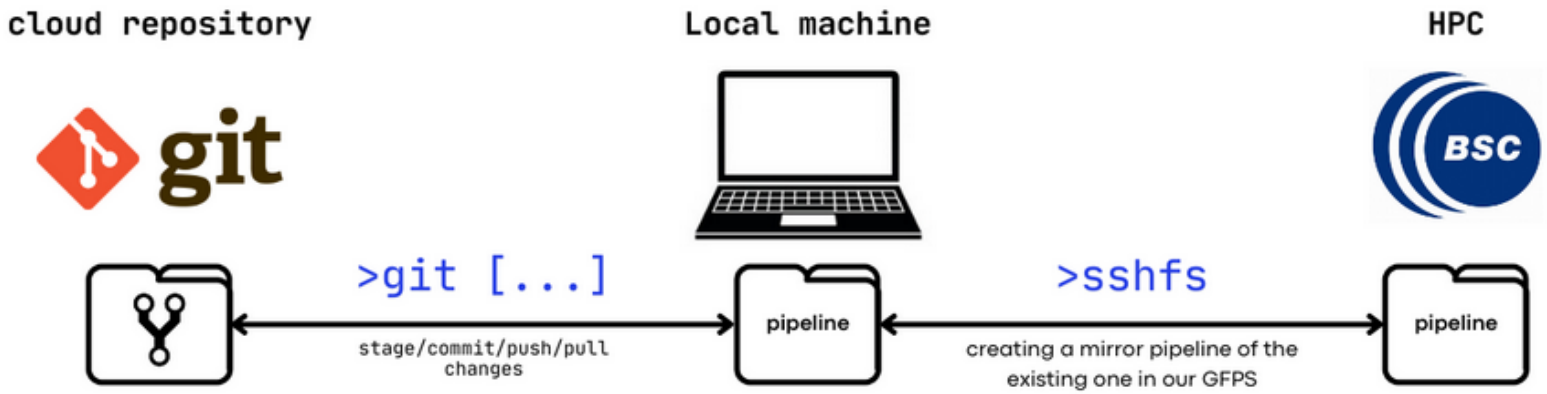
More demanding jobs have more priority, but they need more resources to be available.



1. **Introduction to HPC and BSC-Nord4**
   - Overview of cluster's architecture
   - Nord4 specifications and resources
     - Quotas, data cleaning policies and performance impact
   - SSH, login and file structure
   - Data transfer
2. **Node utilization and resource allocation**
   - CPU and memory requests
   - SLURM
   - How to execute interactive jobs
   - Writing and submitting job scripts
3. **Setting up GitLab**
   - Version control
   - Institutional GitLab
     - SOP (Standard Operational Procedure)
     - Singularity images
4. **Good practices in Nord4**
   - Interactive servers
   - Responsible resource allocation
5. **Practical exercise using FastQC**

Version control

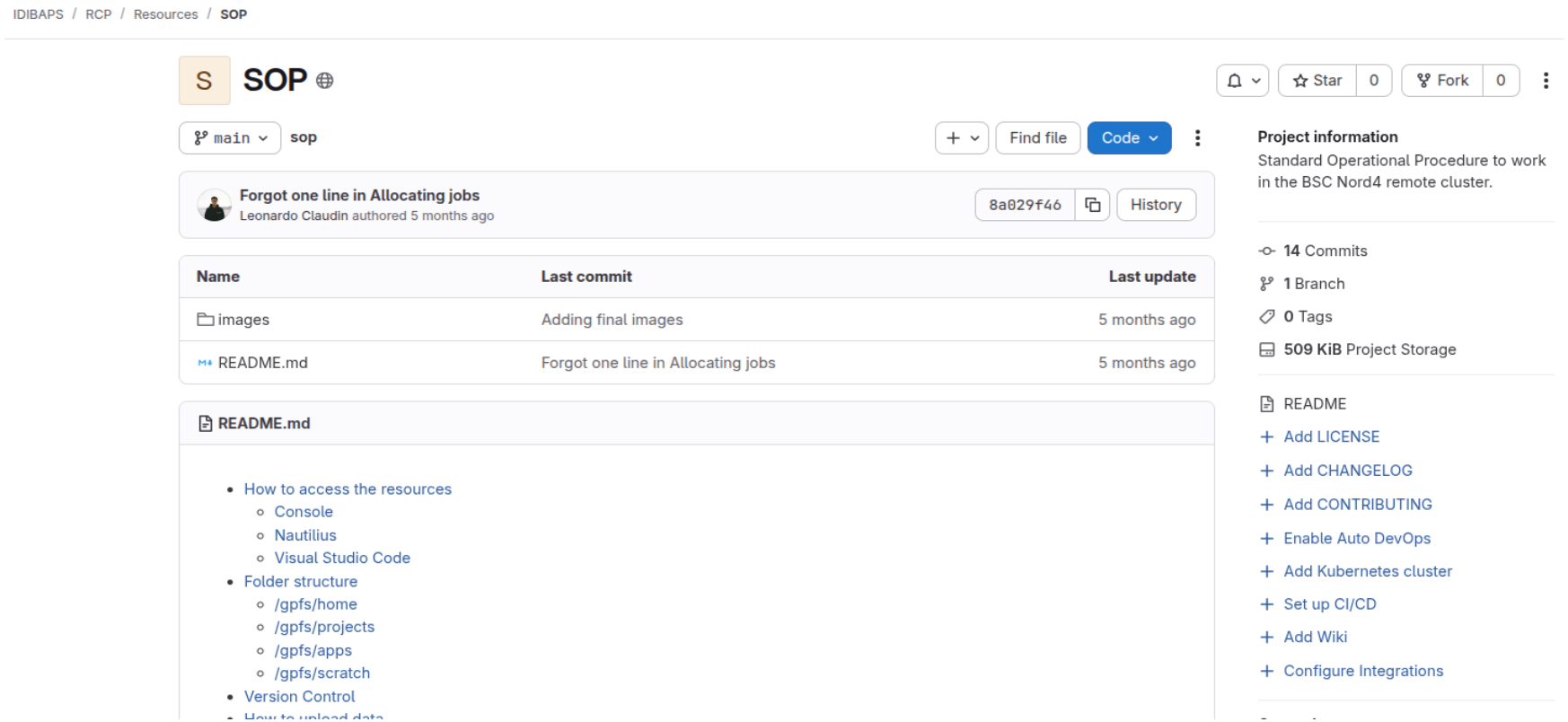
Nord4 has no internet connection.




```
sshfs -o workaround=rename <username>@n<login_node>.bsc.es:/path/to/remote/directory
/path/to/local/empty/directory
```

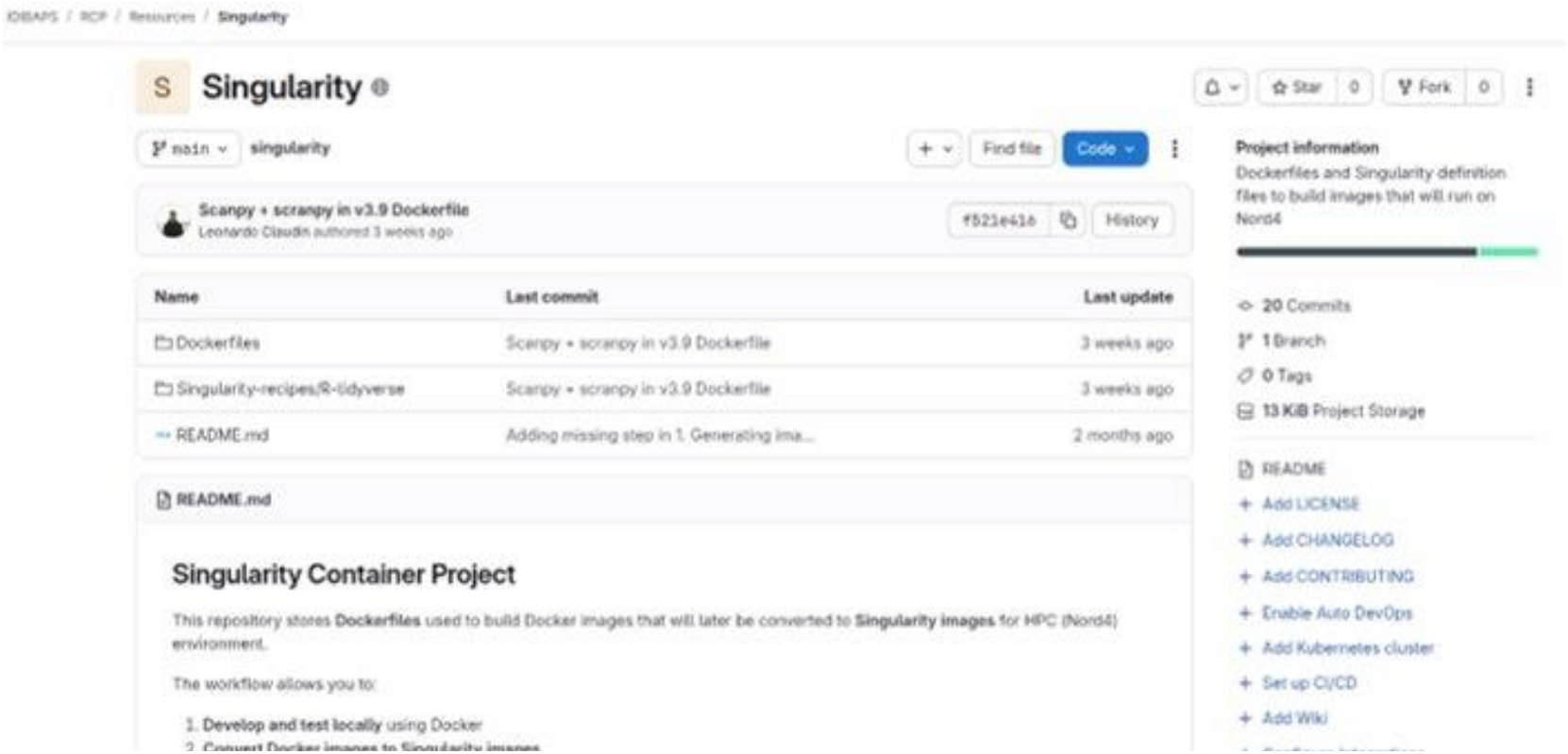
Institutional GitLab

SOP (Standard Operational Procedure)



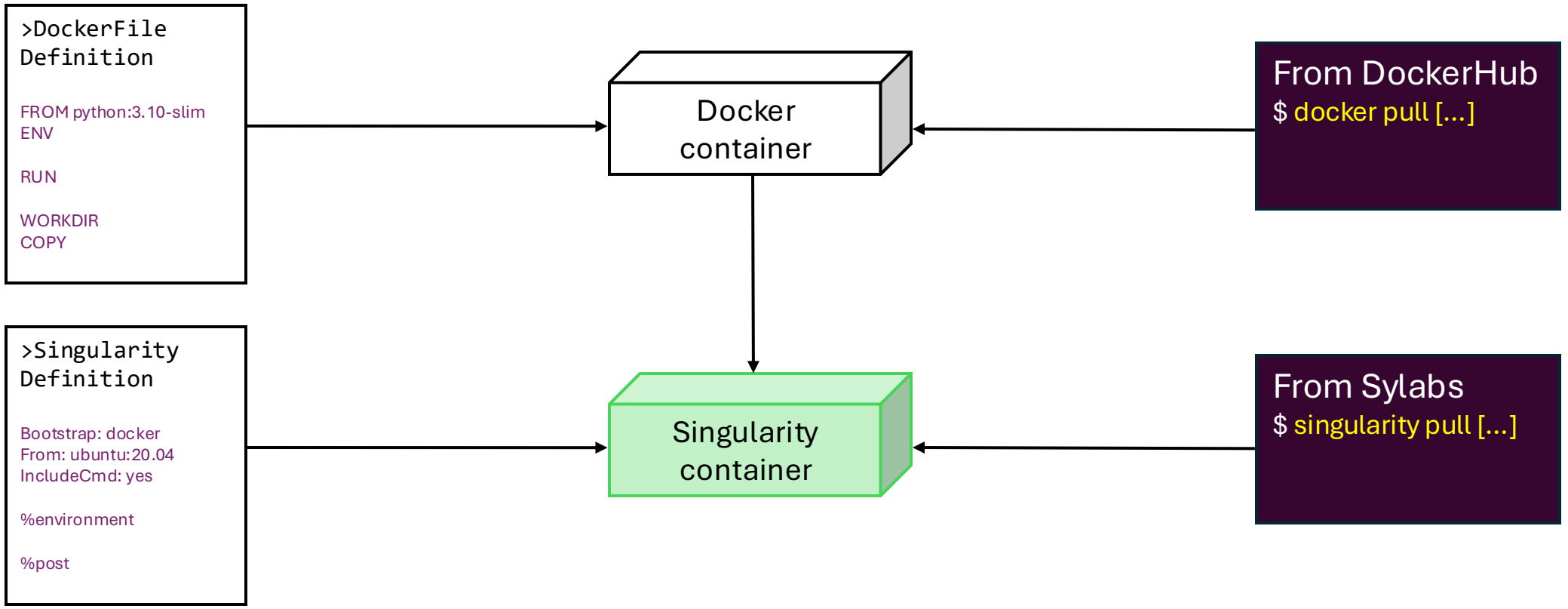
Access to repository here

Singularity: Creating containerized software



Access to repository here

Creating containerized software





1. **Introduction to HPC and BSC-Nord4**
   - Overview of cluster's architecture
   - Nord4 specifications and resources
     - Quotas, data cleaning policies and performance impact
   - SSH, login and file structure
   - Data transfer
2. **Node utilization and resource allocation**
   - CPU and memory requests
   - SLURM
   - How to execute interactive jobs
   - Writing and submitting job scripts
3. **Setting up GitLab**
   - Version control
   - Institutional GitLab
     - SOP (Standard Operational Procedure)
     - Singularity images
4. **Good practices in Nord4**
   - Interactive servers
   - Responsible resource allocation
5. **Practical exercise using FastQC**

Interactive servers



Orientative resources needed

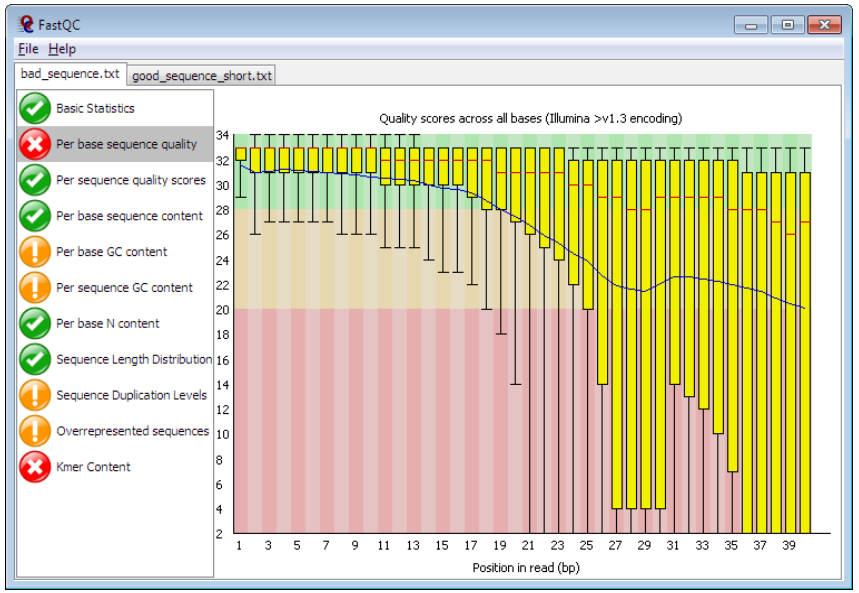
Process	CPUs (recommended — nord4 constraint)	RAM (per sample)	Estimated time (per sample, wall-clock)
alignment (STAR)	8 recommended (would require **≈19 cores** on nord4 to satisfy 38 GB @2GB/core)	38 GB	**~30–60 min**
trimming (fastp/trimmomatic)	4 recommended (ok on nord4: **4 cores** → 8 GB)	8 GB	**~5–15 min**
cellranger (10x, 10k cells)	16 recommended (on nord4 you'd need **≈32 cores** to reach 64 GB @2GB/core; typically request whole node)	64 GB	**~4–8 hours**
freesurfer (recon-all)	8 recommended (on nord4 you'd need **≈16 cores** to reach 32 GB @2GB/core)	32 GB	**~6–12 hours**



1. **Introduction to HPC and BSC-Nord4**
   - Overview of cluster's architecture
   - Nord4 specifications and resources
     - Quotas, data cleaning policies and performance impact
   - SSH, login and file structure
   - Data transfer
2. **Node utilization and resource allocation**
   - CPU and memory requests
   - SLURM
   - How to execute interactive jobs
   - Writing and submitting job scripts
3. **Setting up GitLab**
   - Version control
   - Institutional GitLab
     - SOP (Standard Operational Procedure)
     - Singularity images
4. **Good practices in Nord4**
   - Interactive servers
   - Responsible resource allocation
5. **Practical exercise using FastQC**

FastQC

For quality checks upon FASTQ files



Using four different approaches within Nord4

1. Sequential job
   Using a simple sbatch
2. Parallelized job
   Using a sbatch array
3. Parallelized job
   Using tasks and threads within a simple sbatch
4. Parallelized and containerized job
   Using singularity image

Comments / Doubts ??

RESEARCH COMPUTING email:
research.computing@recerca.clinic.cat

Leo Claudín email:
claudin@recerca.clinic.cat

Miquel Castany email:
mdcastany@recerca.clinic.cat